

CHITSHIT BAYES

Basic Notes

Features: Variable that influences target variables.

Target: Target variables are the variables to be predicted.

1. Determine Likelihood and Prior

Likelihood: Assume the distribution of the data.

a. Normally Distributed (Linear Regression) $[-\infty, \infty]$

If the target variable is numerical (angka), **except** only 0 and 1 then assume the likelihood is normally distributed.

$$Y = \alpha + \sum_{i=1}^n \beta_i x_{ij}$$

With likelihood of $Y_i \sim \text{Normal}(\alpha + \sum_{i=1}^n \beta_i x_{ij}, \sigma^2)$, where X_{ij} being the value for each iteration of j with Prior of $\alpha \sim \text{Normal}(0, 1000)$, $\beta_i \sim \text{Normal}(0, 1000)$, and $\sigma^2 \sim \text{InvGamma}(0.1, 0.1)$.

b. Bernoulli Distribution (Classification) -> Logistic Regression Type 1

if the target variable is **only** 0 and 1, without the number of occurrences of the target variable then assume the likelihood is Bernoulli distributed.

$$\text{logit}[\text{Prob}(Y_i = 1)] = \alpha + \sum_{i=1}^n \beta_i x_{ij}$$

With likelihood of $Y_i \sim \text{Bern}(\text{Prob}(Y_i = 1))$, where $\text{Prob}(Y_i = 1)$ being the value for each iteration of i with Prior of $\alpha \sim \text{Normal}(0, 1000)$ and $\beta_i \sim \text{Normal}(0, 1000)$.

c. Binomial Distribution (Classification) -> Logistic Regression Type 2

If the target variable is only 0 and 1, without the number of occurrences of the target variable then assume the likelihood is Binomial distributed.

Example: Basketball Free throw, Y is the number of successes throw and N is the number of tries, X is the features.

$$Y_i \sim \text{Binomial}(N_i, p_i)$$

Where p_i is the true probability of $Y_i = 1$ using the *logit* function, or can be denoted as

$$\text{logit}(p_i) = \alpha + \sum_{i=1}^n \beta_i x_{ij}$$

With Prior of $\alpha \sim \text{Normal}(0, 1000)$ and $\beta_i \sim \text{Normal}(0, 1000)$.

d. Beta Distribution (Webe Tak ngerti logiknya)

2. Model Fitting: Look at the Photocopy based on the likelihood.

3. Feature Selection: Use SVSS to use the features with the highest *inc_prob*.
4. Model Parameter: To create more than 1 model then change them.
5. Model Validation:
 - a. Check Convergence:
 - i. Plotting: If the shape is like a caterpillar (kaki seribu) then it's convergent.
 - ii. Gelman_diag: Convergent if multivariate score is < 1.1 .
 - b. Check Sample Size (> 1000): Preferably more than 1000.
 - c. Check DIC & WAIC Score: The lower the better, Prioritize WAIC over DIC.
6. Conclusion:

Conclude the best model based on the DIC & WAIC Score.

SEMOGA LULUS.

Linear-Bayes

Wilbert

2024-01-23

Load library

```
library(coda)  
library(rjags)
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
library(knitr)  
library(purrr)  
library(tidyr)  
library(ggplot2)
```

set seed

```
set.seed(123)
```

Load Data

```
df <- read.csv('./ParisHousing.csv')  
head(df)
```

##	squareMeters	numberOfRooms	hasYard	hasPool	floors	cityCode	cityPartRange
## 1	75523	3	0	1	63	9373	3
## 2	80771	39	1	1	98	39381	8
## 3	55712	58	0	1	19	34457	6
## 4	32316	47	0	0	6	27939	10
## 5	70429	19	1	1	90	38045	3
## 6	39223	36	0	1	17	39489	8

##	numPrevOwners	made	isNewBuilt	hasStormProtector	basement	attic	garage	
## 1	8	2005	0		1	4313	9005	956
## 2	6	2015	1		0	3653	2436	128
## 3	8	2021	0		0	2937	8852	135
## 4	4	2012	0		1	659	7141	359
## 5	7	1990	1		0	8435	2429	292
## 6	6	2012	0		1	2009	4552	757

##	hasStorageRoom	hasGuestRoom	price
## 1	0	7	7559082
## 2	1	2	8085990
## 3	1	9	5574642
## 4	0	3	3232561
## 5	1	4	7055052
## 6	0	1	3926647

Model 1

Set Dataframe as matrix and split dependent and independent variable

```
price <- as.matrix(df$price)
Y <- price

X <- df[, c("squareMeters", "numberOfRooms", "hasYard", "hasPool", "floors", "cityCode", "numPrevOwners", "made", "isNewBuilt", "hasStormProtector", "basement", "attic", "garage", "hasStorageRoom", "hasGuestRoom", "cityPartRange")]

names <- c("price", "Square Meters", "Number of Rooms", "Has Yard", "Has Pool", "Number of Floors", "City Code", "Number of Previous Owners", "Year Made", "is Newly Built", "Has Storm Protector", "Basement Area", "Attic Area", "Garage Area", "Has Storage Room", "Has Guest Room", "City Part Range")
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y,X=X,n=n,p=p)
params <- c("alpha","beta")
burn <- 500
n.iter <- 2000
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- inprod(X[i,],beta[])
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Set Initial Value

```
# Set initial value list
inits = list()

# Set Initial value of random value
inits$alpha = rnorm(1)
for(i in 1:p) {
  inits$beta[i] = rnorm(1)
}
inits$taue = 10
```

Compile Model

```
model1 <- jags.model(model_string, data = data, n.chains=n.chains, quiet=TRUE, inits = inits)
```

Update Model

```
update(model1, burn)
```

Get Posterior Samples from the model

```
samples1 <- coda.samples(model1, variable.names=params, n.iter=n.iter, thin=thin)
```

Logistic-Bernoulli-Model

Wilbert

2024-01-23

Load library

```
library(coda)
library(rjags)
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
library(knitr)
library(purrr)
library(tidyr)
library(ggplot2)
```

set seed

```
set.seed(123)
```

Load Data

```
df <- read.csv('./drug200.csv')
head(df)
```

```
##   Age Sex   BP Cholesterol Na_to_K  Drug
## 1  23  F  HIGH         HIGH 25.355 DrugY
## 2  47  M  LOW         HIGH 13.093 drugC
## 3  47  M  LOW         HIGH 10.114 drugC
## 4  28  F NORMAL         HIGH  7.798 drugX
## 5  61  F  LOW         HIGH 18.043 DrugY
## 6  22  F NORMAL         HIGH  8.607 drugX
```

Feature Encoding

Skip aja gak butuh di ujian

```
df$Sex <- as.numeric(factor(df$Sex))
df$BP <- as.numeric(factor(df$BP))
df$Drug <- as.numeric(factor(df$Drug))
df$Cholesterol <- replace(df$Cholesterol, df$Cholesterol=='HIGH', 1)
df$Cholesterol<- replace(df$Cholesterol, df$Cholesterol=='NORMAL', 0)
```

Split Train-Test Data

Skip aja kalau gak disuruh

```
## 70% of the sample size
smp_size <- floor(0.7 * nrow(df))

## set the seed to make your partition reproducible
set.seed(123)
train_ind <- sample(seq_len(nrow(df)), size = smp_size)

train <- df[train_ind, ]
test <- df[-train_ind, ]
```

Set Input Variables

```
Y <- as.matrix(train$Cholesterol)
X <- train[, cbind('Age', 'Sex', 'BP', 'Na_to_K', 'Drug')]
names <- cbind('Age', 'Sex', 'BP', 'Na_to_K', 'Drug')
n <- length(Y)
p <- ncol(X)

burn <- 500
iters <- 2000
chains <- 2
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Scale X

```
X <- as.matrix(scale(X))
```

Modelling

Make Model String

```
mod <- textConnection("model{
# Likelyhood
for(i in 1:n){
  Y[i] ~ dbern(pi[i])
  logit(pi[i]) <- alpha + mu[i]
  mu[i] <- inprod(X[i,], beta[])
  like[i] <- dbin(Y[i],pi[i],1) # For WAIC computation
}

# Prior
alpha ~ dnorm(0,0.01)
for(j in 1:p){
  beta[j] ~ dnorm(0,0.01)
}
}")
```

Fit and Compile Model

```
data <- list(Y=Y,X=X,n=n,p=p)
model <- jags.model(mod,data = data, n.chains=chains,quiet=TRUE)
update(model, burn)
samps <- coda.samples(model, variable.names=c("like"),
n.iter=iters, n.thin = 5)
```

DIC and WAIC

```
# Compute DIC
DIC <- dic.samples(model,n.iter=iters,n.thin = 5)
# Compute WAIC
like <- rbind(samps[[1]],samps[[2]]) # Combine samples from the two chain
fbar <- colMeans(like)
Pw <- sum(apply(log(like),2,var))
WAIC <- -2*sum(log(fbar))+2*Pw
DIC
```

```
## Mean deviance: 192.6
## penalty 6.096
## Penalized deviance: 198.7
```

```
WAIC
```

```
## [1] 199.049
```

Logistic-Binomial-Model

Wilbert

2024-01-23

Load library

```
library(coda)
library(rjags)
```

```
## Linked to JAGS 4.3.1
```

```
## Loaded modules: basemod,bugs
```

```
library(knitr)
library(purrr)
library(tidyr)
library(ggplot2)
```

set seed

```
set.seed(123)
```

Load Data

```
df <- read.csv('./drug200.csv')
head(df)
```

```
##   Age Sex   BP Cholesterol Na_to_K  Drug
## 1  23  F  HIGH         HIGH 25.355 DrugY
## 2  47  M  LOW          HIGH 13.093 drugC
## 3  47  M  LOW          HIGH 10.114 drugC
## 4  28  F NORMAL        HIGH  7.798 drugX
## 5  61  F  LOW          HIGH 18.043 DrugY
## 6  22  F NORMAL        HIGH  8.607 drugX
```

Feature Encoding

Skip aja gak butuh di ujian

```
df$Sex <- as.numeric(factor(df$Sex))
df$BP <- as.numeric(factor(df$BP))
df$Drug <- as.numeric(factor(df$Drug))
df$Cholesterol <- replace(df$Cholesterol, df$Cholesterol=='HIGH', 1)
df$Cholesterol<- replace(df$Cholesterol, df$Cholesterol=='NORMAL', 0)
```

Split Train-Test Data

Skip aja kalau gak disuruh

```
## 70% of the sample size
smp_size <- floor(0.7 * nrow(df))

## set the seed to make your partition reproducible
set.seed(123)
train_ind <- sample(seq_len(nrow(df)), size = smp_size)

train <- df[train_ind, ]
test <- df[-train_ind, ]
```

Set Input Variables

```
Y <- as.matrix(train$Cholesterol)
X <- train[, cbind('Age', 'Sex', 'BP', 'Na_to_K', 'Drug')]
names <- cbind('Age', 'Sex', 'BP', 'Na_to_K', 'Drug')
n <- length(Y)
p <- ncol(X)

burn <- 500
iters <- 2000
chains <- 2
```

Delete Missing Value

```
junk <- is.na(rowSums(X))
Y <- Y[!junk]
X <- X[!junk,]
```

Scale X

```
X <- as.matrix(scale(X))
```

Modelling

Make Model String

```
mod <- textConnection("model{
# Likelyhood
for(i in 1:n){
  Y[i] ~ dbin(pi[i], n)
  logit(pi[i]) <- alpha + mu[i]
  mu[i] <- inprod(X[i,], beta[])
  like[i] <- dbin(Y[i],pi[i],1) # For WAIC computation
}

# Prior
alpha ~ dnorm(0,0.01)
for(j in 1:p){
  beta[j] ~ dnorm(0,0.01)
}
}")
```

Fit and Compile Model

```
data <- list(Y=Y,X=X,n=n,p=p)
model <- jags.model(mod,data = data, n.chains=chains,quiet=TRUE)
update(model, burn)
samps <- coda.samples(model, variable.names=c("like"),
n.iter=iters, n.thin = 5)
```

DIC and WAIC

```
# Compute DIC
DIC <- dic.samples(model,n.iter=iters,n.thin = 5)
# Compute WAIC
like <- rbind(samps[[1]],samps[[2]]) # Combine samples from the two chain
fbar <- colMeans(like)
Pw <- sum(apply(log(like),2,var))
WAIC <- -2*sum(log(fbar))+2*Pw
DIC
```

```
## Mean deviance: 246
## penalty 5.935
## Penalized deviance: 251.9
```

```
WAIC
```

```
## [1] 843.2971
```

Beta regression for the microbiome data

Chapter 4.3.3: Generalized linear models

This model regresses a measure of microbiome species richness onto features of the home from which the sample was taken. Let OTU_{ij} be the abundance of Operational Taxonomic Unit (OTU) j in sample i . The response variable is the proportion of the abundance attributed to the most abundant OTU,

$$Y_i = \frac{\max\{OTU_{i1}, \dots, OTU_{im}\}}{\sum_j OTU_{ij}} \in (0, 1).$$

There are eight covariates (X_{ij}):

1. Longitude
2. Latitude
3. Annual average temperature
4. Annual average precipitation
5. Net primary production
6. Elevation
7. The binary indicator of whether it is a single-family home
8. Number of bedrooms

The regression model is

$$Y_i \sim \text{Beta}(rq_i, r(1 - q_i)) \text{ where } \text{logit}(q_i) = \sum_{l=1}^p X_{il}\beta_l,$$

so that the expected value of Y_i is $q_i \in [0, 1]$ and the concentration around q_i is determined by $r > 0$. The regression coefficients have uninformative priors $\beta_j \sim \text{Normal}(0, 10^2)$ and the concentration parameter has prior $r \sim \text{Gamma}(0.1, 0.1)$.

Load the data

```
set.seed(0820)
```

```
load("S:\\Documents\\My Papers\\BayesBook\\Data\\Microbiome\\homes.RData")
ls()
```

```
## [1] "homes" "OTU"
```

```

city      <- homes[,2]
state     <- homes[,3]
lat       <- homes[,4]
long      <- homes[,5]
temp      <- homes[,6]
precip    <- homes[,7]
NPP       <- homes[,8]
elev      <- homes[,9]
house     <- ifelse(homes[,10]=="One-family house detached from any other h
bedrooms  <- as.numeric(homes[,11])

```

```
## Warning: NAs introduced by coercion
```

```

OTU      <- as.matrix(OTU)
Y        <- apply(OTU,1,max)/rowSums(OTU)
X        <- cbind(long,lat,temp,precip,NPP,elev,house,bedrooms)
names    <- c("Intercept","Longitude","Latitude",
              "Temperature","Precipitation","NPP",
              "Elevation","Single-family home",
              "Number of bedrooms")

# Remove observations with missing values
junk     <- is.na(rowSums(X))
Y        <- Y[!junk]
X        <- X[!junk,]
city     <- city[!junk]
state    <- state[!junk]

# Standardize the covariates
X        <- as.matrix(scale(X))

X        <- cbind(1,X) # add the intercept
colnames(X) <- names
n        <- length(Y)
p        <- ncol(X)

```

Plot the data

```
hist(Y,breaks=50,ylab="Maximum proportion")
```

Fit the beta regression model in JAGS

```
library(rjags)

data <- list(Y=Y,X=X,n=n,p=p)
params <- c("beta","r")

model_string <- textConnection("model{
  for(i in 1:n){
    Y[i] ~ dbeta(r*mu[i],r*(1-mu[i]))
    logit(mu[i]) <- inprod(X[i,],beta[])
  }
  for(j in 1:p){beta[j] ~ dnorm(0,0.01)}
  r ~ dgamma(0.1,0.1)
}")

model <- jags.model(model_string,data = data, n.chains=2,quiet=TRUE)
update(model, 10000, progress.bar="none")
samples <- coda.samples(model, variable.names=params, thin=5, n.iter=20000)

plot(samples)
```

Standardize Covariates

```
X <- as.matrix(scale(X))
```

JAGS

Put Data in JAGS Format

```
n <- length(Y)
p <- ncol(X)

data <- list(Y=Y,X=X,n=n,p=p)
params <- c("alpha","beta")
burn <- 500
n.iter <- 5000
n.chains <- 3
thin <- 5
```

Make Jags Model

```
model_string <- textConnection("model{
  # Likelihood
  for(i in 1:n){
    Y[i] ~ dnorm(alpha+mu[i],taue)
    mu[i] <- theta[i] + inprod(X[i,],beta[])
  }

  # Random Effects
  for(j in 1:n){
    theta[j] ~ ddexp(0,taue)
  }

  # Priors
  for(j in 1:p){
    beta[j] ~ dnorm(0,0.001)
  }

  alpha ~ dnorm(0,0.001)
  taue ~ dgamma(0.1, 0.1)

  # WAIC calculations
  for(i in 1:n){
    like[i] <- dnorm(Y[i],mu[i],taue)
  }

}")
```

Set Initial Value

Plot MCMC Chain Trace and Features Posterior Density

```
par(mar=c(1,1,1,1))
plot(samples1)
```

Descriptive Statistics of model1

```
summary(samples1)
```

Show Alpha and Betas to the corresponding Features

```
sum <- summary(samples1)
rownames(sum$statistics) <- names
rownames(sum$quantiles) <- names
sum$statistics <- round(sum$statistics,3)
sum$quantiles <- round(sum$quantiles,3)
sum
```

Check Convergence

```
gelman.diag(samples1)
```

Compile results

```
ESS1 <- effectiveSize(samples1)
out1 <- summary(samples1)$quantiles
rownames(out1)<-names

ESS1
```

Compute DIC & WAIC

DIC - WAIC : For Model Selection The Lower the better

```
# DIC
dic1 <- dic.samples(model1,n.iter=n.iter)

# WAIC
waic1 <- coda.samples(model1, variable.names=c("like"), n.iter=n.iter)
like1 <- waic1[[1]]
fbar1 <- colMeans(like1)
P1 <- sum(apply(log(like1),2,var))
WAIC1 <- -2*sum(log(fbar1))+2*P1
```

Plot Posterior Density Individually

```
beta <- NULL
for(l in 1:n.chains){
  beta <- rbind(beta,samples1[[l]])
}
colnames(beta) <- names
for(j in 1:17){
  hist(beta[,j],xlab=expression(beta[j]),ylab="Posterior density",
  breaks=100,main=names[j])
}
```

For Feature Selection

```
library(knitr)
Inc_Prob <- apply(beta!=0,2,mean)
Q <- t(apply(beta,2,quantile,c(0.5,0.05,0.95)))
out <- cbind(Inc_Prob,Q)
kable(round(out,2))
```